GNN Practical Assignments

Concepts, Implementation and Results (repository `graphprobe`)

Simachew Alamneh

October 15, 2026

Contents

1 What a Graph Neural Network is **3**

1.1 Message passing . . . 3

1.2 The GCN layer . . . 3

1.3 Node classification and the comparison models . . . 4

2 Repository overview **4**

3 Assignment 1: GCN from scratch **4**

3.1 Goal . . . 4

3.2 Dataset design . . . 4

3.3 Implementation . . . 5

3.4 Correctness checks . . . 5

3.5 Results . . . 5

3.6 Evidence that the model uses graph structure . . . 6

4 Assignment 2: comparing GNN architectures on Cora **6**

4.1 Goal . . . 6

4.2 Data . . . 6

4.3 Implementation . . . 6

4.4 Results . . . 6

4.5 Error analysis and recommendation . . . 6

5 Assignment 3: link prediction and data leakage **7**

5.1 Goal . . . 7

5.2 Implementation . . . 7

5.3 Results . . . 8

5.4 Why the leaky evaluation is invalid . . . 8

5.5 Unobserved links are not confirmed negatives . . . 8

6 Assignment 4: MLOps pipeline **8**

6.1 Goal . . . 8

6.2 Operations, one by one . . . 9

6.3 Gate rules . . . 10

6.4 Serving unseen nodes . . . 10

6.5 Drift detection . . . 10

6.6 Failure handling . . . 11

7 Reproduction, limitations and honest notes **11**

1 What a Graph Neural Network is

A *graph* $G = (V, E)$ has nodes V and edges E. Each node i carries a feature vector $x_i \in \mathbb{R}^F$, and some nodes have a class label. The task in all four assignments is to use both the features and the edges.

A standard classifier such as a multilayer perceptron (MLP) maps x_i to a class score using node i alone. A graph neural network (GNN) first combines x_i with the features of the neighbours of i, and then classifies the combined vector. This helps when connected nodes tend to share a class, a property called *homophily*. The edge homophily of a graph is the fraction of edges whose two endpoints have the same class.

1.1 Message passing

One GNN layer has three steps for every node i:

1. **Message:** each neighbour j (and i itself) sends its transformed features $x_j W$.
2. **Aggregate:** node i combines the incoming messages, here by a weighted sum.
3. **Update:** add a bias and apply a nonlinearity.

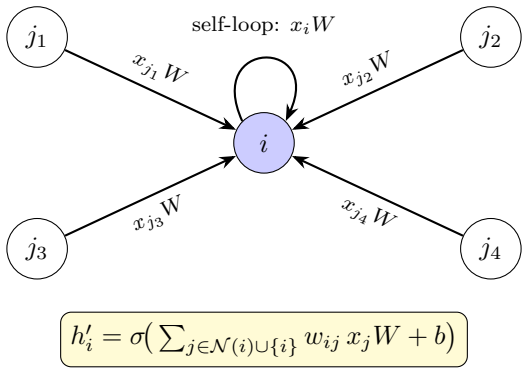


Figure 1: One message-passing step. Neighbours and the node itself send messages to node i; the weighted sum is the new representation of i.

1.2 The GCN layer

The graph convolutional network (GCN) layer uses fixed weights from node degrees:

$$H^{(l+1)} = \sigma\big(\hat{A}\, H^{(l)} W^{(l)} + b^{(l)}\big), \qquad \hat{A} = D^{-1/2}(A + I)D^{-1/2}. \tag{1}$$

Here A is the adjacency matrix, I adds a self-loop to every node, and D is the diagonal degree matrix of $A + I$. For one edge from j to i the weight is

$$w_{ij} = \frac{1}{\sqrt{d_i\, d_j}}. \tag{2}$$

Self-loops keep the node's own features in its aggregate. **Degree normalisation** stops nodes with many neighbours from producing much larger sums than nodes with few neighbours. A two-layer GCN lets each node use information from nodes up to two edges away.

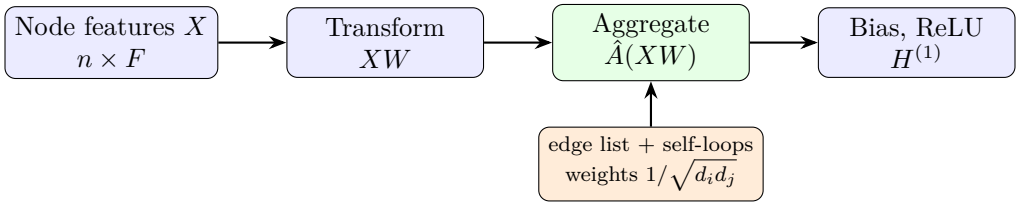


Figure 2: Computation of one GCN layer. The aggregation uses the edge list with self-loops and the degree weights.

1.3 Node classification and the comparison models

For node classification, two GCN layers map $F \to h \to C$ class scores, trained with cross-entropy on the labelled training nodes. The experiments compare:

Model	How neighbours are used
MLP	Not used. It is the control that shows what features alone achieve.
GCN	Fixed degree-based averaging.
GraphSAGE	Mean of neighbours, with a separate weight for the node itself.
GAT	Learned attention weights for each neighbour.

2 Repository overview

Folder	Assignment	Entry point
`scratch_gcn/`	GCN from scratch and verification	`python -m scratch_gcn.run`
`cora_benchmark/`	MLP, GCN, GraphSAGE, GAT on Cora	`python -m cora_benchmark.run`
`link_prediction/`	Link predictor and leakage study	`python -m link_prediction.run`
`mlops/`	Tracked, served, gated, monitored model	`python -m mlops.cli -help`
`tests/`	Automated tests (35)	`pytest -q`

3 Assignment 1: GCN from scratch

3.1 Goal

Build and verify a two-layer GCN using only PyTorch tensor operations, and show whether it uses graph structure.

3.2 Dataset design

The generator `make_dataset` in `scratch_gcn/core.py` builds the data so that edges add information beyond the features.

- **Nodes and labels:** 200 nodes, two balanced classes, shuffled.
- **Features (3 per node):** features 1 and 2 are the class mean (± 0.6) plus Gaussian noise $\mathcal{N}(0, 1)$. Feature 3 is pure noise. The classes overlap, so features alone are only weakly informative.
- **Edges:** a stochastic block model. Two nodes of the same class connect with probability 0.08, nodes of different classes with probability 0.01. This gives 897 undirected edges, of which 88.6% join same-class nodes.

- **Split:** random 120 training, 40 validation, 40 test nodes. Labels are never part of the input features.

3.3 Implementation

Requirement	Code
Self-loops	`prep_graph` appends (i, i) for every node (1794 $\rightarrow$ 1994 messages).
Degree normalisation	`deg` counts incoming messages; `norm` = `deg[src]`$^{-1/2}$`deg[dst]`$^{-1/2}$.
Message passing	`GCNLayer`: `h = x @ W`; `index_add_(0, dst, norm[:,None]*h[src])` sums weighted messages at each receiver.
Two-layer GCN	`GCN`: layer $3 \rightarrow 16$, ReLU, dropout, layer $16 \rightarrow 2$.
Training loop	`train_model`: cross-entropy on training nodes, `backward()`, Adam step; keeps the checkpoint with the best validation accuracy.
No ready-made layers	Only `torch` tensor operations are used.

The forward pass runs over all nodes, but the loss uses only the 120 training labels. Validation and test labels are never used for gradients.

3.4 Correctness checks

1. **Manual calculation:** for one node, the layer output is recomputed with Python loops from the neighbour list, degrees and the same weights. Maximum absolute difference: 1.2×10^{-7}.
2. **Reordering:** nodes, features and edges are relabelled consistently and the edge order is shuffled. The outputs equal the original outputs with rows reordered. Maximum difference: 9.5×10^{-7}.

The differences are floating-point rounding, far below the 10^{-5} tolerance.

3.5 Results

Setting	Test accuracy (5 seeds)
MLP, features only	0.840 ± 0.022
GCN, true graph	0.975 ± 0.000
GCN, 25% edges removed at test time	0.975 ± 0.000
GCN, 50% removed	0.975 ± 0.018
GCN, 75% removed	0.915 ± 0.038
GCN, 100% removed	0.825 ± 0.000
GCN retrained on a random graph (same edge count)	0.720 ± 0.108

Table 1: Scratch GCN ablations. The test set has only 40 nodes, so differences of a few points are noise.

Edge homophily	GCN	MLP	GCN − MLP
0.887	1.000	0.817	+0.183
0.802	0.983	0.817	+0.167
0.664	0.867	0.817	+0.050
0.567	0.650	0.817	−0.167
0.495	0.617	0.817	−0.200

Table 2: Homophily sweep: the edge probability p_{out} is raised from 0.01 to 0.08 while the features stay fixed.

3.6 Evidence that the model uses graph structure

The GCN beats the MLP on identical features and split. Removing edges at test time lowers its accuracy to roughly the MLP level. A random graph hurts it badly. In the sweep, the GCN advantage shrinks as edges lose class information and reverses when edges are uninformative. Together these show that the improvement comes from informative edges.

4 Assignment 2: comparing GNN architectures on Cora

4.1 Goal

Compare an MLP, GCN, GraphSAGE and GAT on Cora, study depth and edge removal, analyse errors, and recommend a model from evidence.

4.2 Data

Cora has 2708 documents, 1433 binary word features (about 18 words per document), 7 classes, 5278 undirected citation links, mean degree 3.9 and edge homophily 0.81. The standard split has 140 training, 500 validation and 1000 test nodes.

4.3 Implementation

- `models.py`: one `Net` class builds any of the four models with any depth, using library layers (`GCNConv`, `SAGEConv`, `GATConv`). The last layer outputs class scores.
- `training.py`: own training loop and evaluation. Best checkpoint chosen by validation accuracy, early stopping after 50 epochs without improvement. Test metrics are computed only when `report_test=True`.
- `tune.py`: equal budget. Every model receives the same 12 random configurations (a fixed sampler seed makes the draws identical), the same epochs and the same tuning seed. Selection uses validation accuracy only.
- `run.py`: three seeds per model; depth study with 1, 2, 4, 6, 8 layers; edge-removal study with 0–100% of edges removed from the training and test graph; error analysis.

4.4 Results

4.5 Error analysis and recommendation

For the best GNN (GCN), the error rate is 0.516 on test nodes whose neighbours mostly belong to other topics, against 0.105 on the rest. Misclassified nodes include low-degree nodes whose only

Model	Test accuracy	Macro F1	Train time (s)	Parameters
MLP	0.596 ± 0.010	0.575 ± 0.010	12.0	92,231
GCN	0.825 ± 0.008	0.811 ± 0.007	13.6	23,063
GraphSAGE	0.799 ± 0.004	0.790 ± 0.005	31.2	92,199
GAT	0.800 ± 0.016	0.790 ± 0.015	12.4	23,109

Table 3: Cora results, mean ± standard deviation over 3 seeds. Hyperparameters were selected on validation accuracy.

GCN layers	1	2	4	6	8
Test accuracy	0.780	0.825	0.539	0.339	0.282
Mean pairwise cosine of embeddings	0.36	0.68	0.71	0.88	0.998

Table 4: Depth study for GCN. A cosine near 1 means node embeddings become nearly identical (over-smoothing). The MLP also degrades with depth, so part of the loss is training difficulty.

neighbour has another label, and high-degree nodes whose neighbourhood is mostly correct but whose own features point elsewhere. The MLP is correct on 25% of the sampled GCN errors.

Recommendation: GCN. It has the highest accuracy and F1, the fewest parameters among the strong models, and short training time. Its lead over GraphSAGE and GAT is about 2.5 points on 3 seeds, so it is a modest lead. Two layers are best; deeper models fail.

5 Assignment 3: link prediction and data leakage

5.1 Goal

Predict missing citation links on Cora without class labels, and show how leakage inflates scores.

5.2 Implementation

1. **Split:** the 5278 unique undirected links are split into 4488 train, 263 validation and 527 test links. Each link is stored once ($u < v$), so both directions stay in the same split.
2. **Negatives:** pairs with no link, excluding self-links and *all* observed links. Validation and test negatives are fixed once and do not overlap. Training negatives are resampled every epoch and avoid the evaluation negatives.
3. **Message passing:** the encoder reads only training links in training, validation and testing.
4. **Encoder:** a two-layer GCN producing 64-dimensional node embeddings z.
5. **Dot decoder:** $s(u, v) = z_u^\top z_v$.
6. **MLP decoder:** a small network on $[z_u \odot z_v,\ |z_u - z_v|]$. Both parts are unchanged when u and v swap, so reversed pairs get the same score by construction (maximum difference 0.0).
7. **Baseline:** common neighbours computed from training links.
8. **Automated checks:** eleven checks confirm that held-out links and their reverses are absent from the message-passing graph, splits are disjoint, and negatives are valid. The same checker fails on the leaky graph, which shows that it detects leaks.

Edges removed	0%	25%	50%	75%	90%	100%
MLP (control)	0.596	0.596	0.596	0.596	0.596	0.596
GCN	0.825	0.800	0.752	0.685	0.619	0.559
GraphSAGE	0.799	0.749	0.701	0.615	0.584	0.608
GAT	0.794	0.765	0.716	0.559	0.451	0.344

Table 5: Test accuracy when a fraction of edges is removed from both training and test graph.

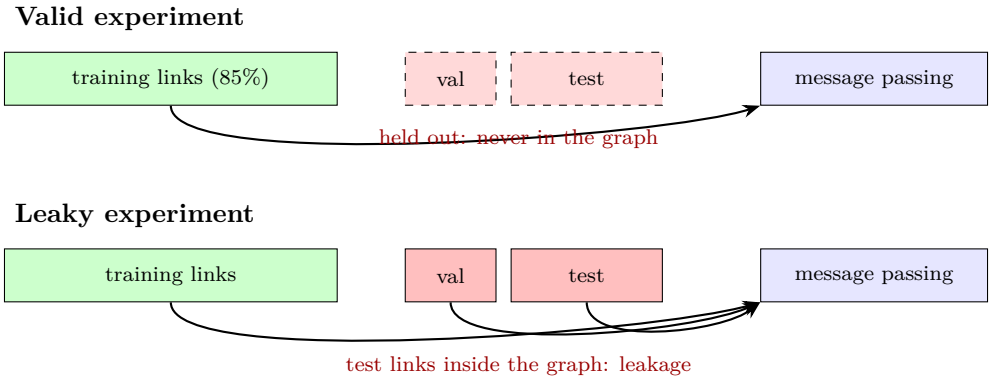


Figure 3: Valid pipeline against the deliberately leaky experiment.

5.3 Results

5.4 Why the leaky evaluation is invalid

In the leaky graph the test link (u, v) is itself an edge. The encoder averages over neighbours, so u receives v's features and v receives u's, which makes their embeddings similar and the pair scores high. The model reads the answer from the graph instead of predicting an unseen link. A real missing link is, by definition, not in the graph.

5.5 Unobserved links are not confirmed negatives

Cora is an incomplete sample of citations. A pair without a recorded link may be a real relationship that was never recorded, so a high-scoring unobserved pair may be correct, and a high-scoring "false positive" may not be an error. Predicted candidates need outside validation. Link predictors also tend to favour high-degree nodes, which the top-10 candidate list shows.

6 Assignment 4: MLOps pipeline

6.1 Goal

Turn the GCN into a reproducible, tracked, deployable system that is safe to update. The model is the scratch GCN on Cora. A set of 150 test nodes is held out as *unseen* nodes: they are removed from the training graph together with their incident edges (1,030 directed edge entries) and introduced at inference time.

Setting	Method	Test ROC-AUC	Test AP
valid	common neighbours	0.709	0.707
valid	GCN + dot	0.928	0.934
valid	GCN + MLP	0.942	0.949
leaky, eval only	GCN + dot / MLP	0.974 / 0.996	0.972 / 0.995
leaky, train + eval	GCN + dot / MLP	0.975 / 0.996	0.973 / 0.995
leaky	common neighbours	0.762	0.759

Table 6: Link prediction on the held-out test links (3 seeds for learned models). Leaky rows are never used for model selection.

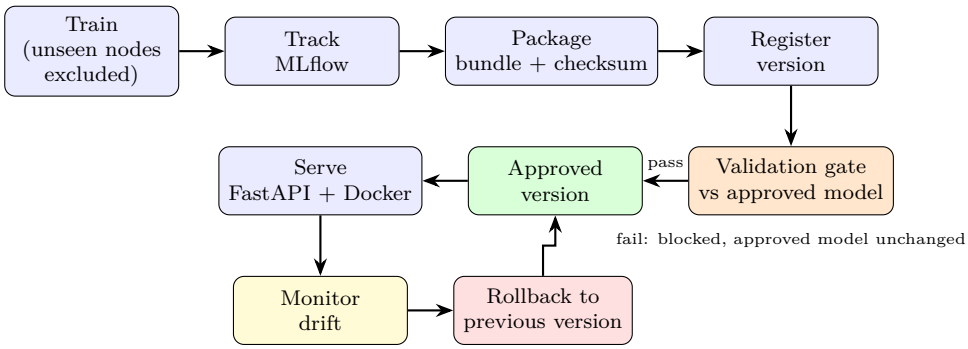


Figure 4: The MLOps workflow. CI runs the tests and the gate on every push.

6.2 Operations, one by one

1. **Untracked baseline** (`baseline_untracked.py`): a bare training script. Its limitations: no recorded seed, hyperparameters, code or dataset version; metrics are printed and lost; nothing is saved, so it cannot be served; unseen nodes are not excluded; there is no gate, rollback, test or monitoring.

2. **Experiment tracking** (`train.py`, MLflow): each run logs hyperparameters, seed, validation and test metrics, git commit, code hash, dataset hash and the model bundle. Three experiments were recorded, with validation macro-F1 of 0.7940, 0.7941 and 0.7902.

3. **Model packaging** (`bundle.py`): six files: weights, architecture, preprocessing (row normalisation), feature schema (1433 features), label map and metadata with a SHA-256 checksum of the weights. Loading verifies the checksum. Metrics are computed from the reloaded bundle, and a test verifies that reloading reproduces predictions exactly.

4. **API and Docker** (`service.py`, `Dockerfile`): `POST /predict` accepts features, edges and target node IDs and returns each prediction with class name, probabilities, an isolated-node flag and the model version. `GET /health`, `GET /model` and `POST /admin/reload` complete the service. The Docker image uses CPU-only PyTorch.

5. **Registry and validation gate** (`registry.py`): a file-based registry stores versions and the approved version. At promotion time the candidate and the approved model are both scored on the same validation set, so no stale stored number is compared.

6. **CI and rollback**: GitHub Actions runs the tests and `gate-check` on every push; the check exits with code 1 if the candidate regresses by more than the tolerance, which fails the job. Rollback restores the previous approved version, and `/admin/reload` makes the running service use it.

7. **Drift detection** (`drift.py`): four statistics compare two data snapshots using a fixed model.
8. **Failure handling**: each invalid input has a defined response (Table 7).
9. **Prediction consistency**: offline predictions and API responses on identical inputs had identical labels and a maximum probability difference of 0.

6.3 Gate rules

The monitored metric is validation macro-F1. Rules are applied in this order:

#	Condition	Decision
1	Candidate metric missing or NaN	Blocked
2	No approved model (first model)	Promoted only if metric ≥ 0.70
3	Approved model's metric missing	Blocked (fail closed)
4	Exact tie	Blocked (configurable)
5	Improvement ≥ 0.005	Promoted
5	Smaller gain, or a decrease	Blocked

In the gate demonstration a weak first model (0.735) was promoted, a better model was promoted ($+0.059$), an undertrained model was blocked (-0.464), an exact tie was blocked, a missing metric was blocked, and rollback restored the first version.

6.4 Serving unseen nodes

Case	Accuracy
Existing nodes, training-graph context	0.794
Unseen nodes, features and all their edges supplied	0.847
Unseen nodes, half of their edges missing	0.767
Unseen nodes, isolated (features only)	0.727

Limitation: every request must contain the target's features, its neighbours' features and the edges, because a GCN needs graph context, and degree normalisation depends on the supplied graph. Accuracy falls as context is removed.

6.5 Drift detection

Four statistics are compared between a reference snapshot and a new snapshot: relative change of mean degree and absolute change of the isolated-node fraction (graph structure), and cosine distance of the embedding centroid and relative change of the mean embedding norm (embeddings). Thresholds come from the data: random half-splits of the baseline data give the natural variation of each statistic, and the alert threshold is its mean plus three standard deviations.

Scenario	Alerts	Example value (threshold)
New random sample of the same data	none	degree change 0.074 (0.241)
50% of edges removed	degree, isolated fraction, norm	degree change 0.463 (0.241)
60% of vocabulary columns removed	centroid shift, norm	centroid distance 0.0196 (0.0013)

Embeddings from different model versions are not directly comparable. Hidden units have no fixed meaning: a different seed or version produces a rotated and permuted coordinate system. For two seeds the correlation between same-index coordinates was only 0.34, while linear CKA, which is invariant to rotation and permutation, was 0.93. Version comparisons should use such measures instead of raw coordinates.

6.6 Failure handling

Case	Status	Behaviour
Isolated target node	200	Prediction from own features, `isolated=true`, warning
Duplicate target IDs	422	`duplicate_target_ids`
Invalid or negative edge endpoint	422	`invalid_edge_endpoint`
Wrong feature dimension	422	`bad_feature_dim`
Unknown target ID	404	`unknown_target_id`
Empty target list	422	`empty_targets`
Duplicate edges, self-loops	200	Cleaned silently
Missing, corrupt or unapproved model	503	`model_unavailable`

Table 7: Expected behaviour for each failure case.

7 Reproduction, limitations and honest notes

Commands.

Setup	`pip install torch`; `pip install -r requirements.txt`; `pytest -q`
Assignment 1–3	`python -m scratch_gcn.run`; `python -m cora_benchmark.run`; `python -m link_prediction.run`
Assignment 4	`python -m mlops.cli experiments`; `gate-demo`; `unseen-demo`; `drift-demo`; `consistency-demo`; `failure-demo`; `gate-check`; `serve`

Significant implementation problem. A long Cora run stopped partway and left only partial result files. The missing edge-removal results and the absence of a running process showed that the job had been killed. Because the tuning step is deterministic, resume logic was added that reuses the saved best configurations and the saved depth study, and only the remaining steps were rerun. A second problem: the first gate demonstration used a weak first model below the 0.70 floor, so nothing was approved and the rollback step failed; a weaker but valid configuration (validation macro-F1 0.735) fixed it. *Replace this section with a problem you personally investigated if you have one.*

Limitations.

- The scratch GCN test set has only 40 nodes, so small differences are noise.
- Cora results use 3 seeds, and the GCN lead over GraphSAGE and GAT is about 2.5 points.
- Link prediction uses one split seed.
- The Docker image and the GitHub Actions workflow were not run in the authoring environment.

- The registry is a custom file-based implementation, chosen so that gate and rollback rules are explicit, testable and runnable in CI; MLflow is used for tracking only.